

```
document.write("<pre>");
document.write("Starting with m = 1, n = 1:<br />");
document.write(ackermann(1, 1) + "<br />");
document.write("Starting with m = 2, n = 3:<br />");
document.write(ackermann(2, 3) + "<br />");
document.write("</pre>");
</script>
```

When you run this code, the output's indentation (set by the `indentation` argument) tells you how deep on the call stack the given recursive function call is:

```
Starting with m = 1, n = 1:
ackermann(1, 1)
  ackermann(1, 0)
    ackermann(0, 1)
      ackermann(0, 2)
        3
Starting with m = 2, n = 3:
ackermann(2, 3)
  ackermann(2, 2)
    ackermann(2, 1)
      ackermann(2, 0)
        --snip--
          ackermann(0, 6)
            ackermann(0, 7)
              ackermann(0, 8)
                9
```

You can also try `ackermann(3, 3)`, but anything with larger arguments will probably take far too long to calculate. To speed up the calculation, try commenting out all `print()` and `document.write()` calls except the ones that print the final return value of `ackermann()`.

Remember, even a recursive algorithm like the Ackermann function can be implemented as an iterative function. The iterative Ackermann algorithms are implemented in *ackermannIterative.py* and *ackermannIterative.html* in the downloadable resources at <https://nostarch.com/recursive-book-recursion>.